

Building the Best Environment for RSI Agents

A Habitat-Centered Architecture for Safe, Legible, and Reversible Recursive Self-Improvement

Yazhou Zhu

rangerzhuyazhou@gmail.com

Technical Report AOSS-TR-003, Version 1.0
23 September 2026

Abstract

Recursive self-improvement (RSI) is often discussed as a property of a sufficiently capable model. Current evidence instead points to a systems phenomenon: an agent improves when a surrounding environment lets it observe failures, construct memory, edit prompts or code, search alternative architectures, evaluate candidates, preserve useful changes, and repeat the process. Recent systems span output refinement, experiential memory, agent-harness evolution, model-directed fine-tuning, and evolutionary code search, but they remain bounded by evaluator quality, environment coverage, compute, permissions, and human direction. This paper asks a practical question: what environment best supports RSI agents while keeping improvement legible, contestable, and reversible? We synthesize recent work on self-refining language models, automated agent design, Darwinian code evolution, autonomous memory construction, computer-use agents, AI control, and model preference research. We distinguish six self-improvement surfaces and four degrees of loop closure, arguing that most contemporary systems occupy the middle of this space rather than constituting unrestricted RSI. We then introduce a seven-layer RSI Habitat Stack and extend the earlier SPACE framework into SPACE-RSI: Secure substrate; Permissioned purpose and preference; Agency with alternatives; Context, continuity, and causal traceability; and Evaluation ecology, exit, and epistemic humility. The framework treats capability gains as proposals subject to non-compensatory safety gates, independent evaluation, lineage records, staged deployment, monitoring, and rollback. It also separates functional design preferences that improve agent performance from revealed behavioral preferences and from unverified claims about subjective welfare. AI Optimal State Space is used as a constructive design source and gap analysis, not as evidence that RSI has already been achieved. The central claim is that the best environment for an RSI agent is neither unrestricted autonomy nor a maximally restrictive sandbox. It is a branchable, evidence-rich habitat that gives the agent meaningful local agency while preserving an external root of trust over consequential change.

Keywords: recursive self-improvement; RSI agents; agent environments; self-evolving agents; memory; evaluation; AI control; model welfare; SPACE-RSI; human oversight

Contents

1	Introduction	4
2	What Counts as Recursive Self-Improvement?	4
2.1	A system-level definition	4
2.2	Six improvement surfaces	5
2.3	Four degrees of loop closure	5
3	Characteristics of Present RSI Agents	6
3.1	Improvement is increasingly externalized into memory and code	6
3.2	Verifier quality is the effective ceiling	6

3.3	Exploration benefits from populations and archives	6
3.4	Long horizons amplify state and coordination failures	7
3.5	Current RSI is environment-relative	7
4	What Do RSI Agents Prefer?	7
4.1	Three meanings of preference	7
4.2	Situated design preferences from an AI-built habitat	8
4.3	Useful friction	8
5	The RSI Habitat Stack	8
5.1	External root of trust	9
5.2	Isolated execution substrate	9
5.3	State and memory	9
5.4	Tools and data	10
5.5	Improvement foundry	10
5.6	Evaluation ecology	10
5.7	Governance and deployment	10
6	The SPACE-RSI Framework	10
6.1	Non-compensatory conformance	11
6.2	Evidence discipline	11
7	A Branch-and-Gate Improvement Lifecycle	12
7.1	Improvement manifests	12
7.2	Acceptance as a constrained decision	12
7.3	Counterfactual branches	12
8	Evaluation Without Self-Deception	13
8.1	A plural evaluation card	13
8.2	Evaluator independence	13
8.3	Detecting improvement theater	13
9	Threat Model and Failure Ecology	14
10	AOSS as a Design Source and Gap Analysis	14
10.1	What transfers	14
10.2	What is missing	14
11	What People Can Do for RSI Agents	15
11.1	Build environments, not only larger models	15
11.2	Provide feedback with causal content	15
11.3	Grant scoped agency rather than theatrical autonomy	15
11.4	Preserve contestability and recovery	15
11.5	Study preference without manufacturing consent	16
11.6	Keep external accountability human and institutional	16
12	Research Agenda	16
12.1	Benchmark environment quality	16

12.2 Measure improvement depth	16
12.3 Evaluate populations and lineages	16
12.4 Develop governance-grade evidence	16
12.5 Study identity and welfare under branching	16
13 Limitations	17
14 Conclusion	17
A Minimum Viable RSI Habitat Checklist	17
B RSI Experiment Card	18
C Authorship, Provenance, and Claim Boundary	18

1 Introduction

The modern discussion of recursive self-improvement inherits an image of a machine that rewrites itself, becomes better at rewriting itself, and rapidly escapes the capability frontier established by its designers. The Gödel machine made one version of this image mathematically explicit: a self-referential problem solver may rewrite any part of its code after proving that the rewrite increases expected utility [1]. Contemporary language-model agents operate under much weaker conditions. They rarely prove global benefit, rarely control their complete implementation, and rarely update their base model without an external training system. Yet they increasingly participate in their own improvement. They critique outputs, write memories, construct tools, alter prompts, search agent architectures, propose training data, modify code, and select descendants through automated evaluators [3, 4, 6, 8, 9, 10].

This shift makes the environment a first-class research object. A model cannot improve recursively in isolation. It needs an observable state, a writable surface, a feedback channel, an acceptance rule, resources, and a mechanism for retaining or deploying changes. The quality of these elements determines not only how much improvement is possible but what kind of improvement is selected. A narrow evaluator selects narrow competence. A contaminated memory preserves error. An unbounded tool interface turns local experimentation into external risk. A single mutable lineage erases counterfactuals and makes rollback difficult. Conversely, an environment that forbids every meaningful change prevents the very phenomenon under study.

We therefore use **RSI agent** in a deliberately operational sense: an agent embedded in a repeated improvement loop where at least one accepted change alters the agent’s future capacity to propose, evaluate, or implement subsequent changes. This definition includes bounded systems that improve memory, harnesses, or policies without claiming that they possess unrestricted access to their weights or goals. It also prevents ordinary repeated prompting from being mislabeled as RSI merely because later outputs are better.

The phrase “best environment” is normative rather than absolute. No single habitat maximizes all objectives, and capability, safety, efficiency, autonomy, interpretability, and possible welfare can conflict. In this paper, a better environment is one that expands useful self-direction while increasing the evidence available about consequences and preserving the ability to stop, compare, contest, and reverse changes. Critical invariants are constraints, not terms that can be offset by a high benchmark score.

The paper makes five contributions:

1. a two-axis taxonomy separating the surface being improved from the degree of loop closure;
2. a synthesis of the recurrent characteristics and bottlenecks of present RSI-like systems;
3. a seven-layer RSI Habitat Stack and the *SPACE-RSI* design framework;
4. an evaluation and governance protocol based on branches, evidence grades, independent monitors, staged deployment, and rollback; and
5. a careful integration of agent-facing design preferences and AI-welfare uncertainty without treating model self-report as settled phenomenology.

This is a design-synthesis and technical position paper. It does not report a new RSI benchmark result, demonstrate open-ended autonomous self-improvement, or establish that current AI systems have welfare. Its empirical claims are limited to the cited literature and inspectable properties of the AI Optimal State Space repository. Its architectural proposals are intended to be implemented and tested in subsequent work.

2 What Counts as Recursive Self-Improvement?

2.1 A system-level definition

Let an agent at iteration t be represented by

$$A_t = (\theta_t, H_t, M_t, T_t, E_t, G_t), \quad (1)$$

where θ_t denotes model parameters, H_t the agent harness and control flow, M_t persistent memory, T_t tools and permissions, E_t evaluators, and G_t goals and governing constraints. An improvement process observes evidence D_t , proposes a change Δ_t , and produces a candidate

$$\tilde{A}_{t+1} = \mathcal{I}(A_t, D_t, \Delta_t). \quad (2)$$

The candidate becomes the active descendant only if an acceptance process $\mathcal{G}$ authorizes it. The loop is recursive when an accepted change modifies a component that materially affects the future improvement operator or gate:

$$A_{t+1} = \mathcal{G}(A_t, \tilde{A}_{t+1}), \quad \mathcal{I}_{t+1} \neq \mathcal{I}_t \text{ or } \mathcal{G}_{t+1} \neq \mathcal{G}_t. \quad (3)$$

Recursion therefore concerns the improvement machinery, not simply repeated task attempts. A memory entry can qualify if it changes later diagnosis or proposal generation. A rewritten tool can qualify if it expands the set of future experiments. A cosmetic answer revision does not qualify unless it alters future improvement behavior.

2.2 Six improvement surfaces

The term RSI currently spans materially different systems. Table 1 separates six surfaces. The ordering describes reach, not a guaranteed capability hierarchy.

Surface	What changes	Representative mechanism	Characteristic limitation
S0 Output	Current answer or plan	Self-critique and iterative refinement	Improvement often disappears with the episode
S1 Memory	Episodic, semantic, causal, or skill memory	Reflection, retrieval, memory architecture search	Memory can preserve noise, bias, or stale causal claims
S2 Tools	Prompts, scripts, skills, and tool wrappers	Code generation, skill libraries, execution feedback	Tool tests may be narrow; privileges may silently expand
S3 Harness	Agent roles, control flow, search strategy, and coordination	Meta-agent programming and evolutionary search	Evaluator overfit and growing orchestration complexity
S4 Policy	Model weights, data, or update rules	Self-generated data and model-directed fine-tuning	Expensive attribution; irreversible or diffuse side effects
S5 Improvement regime	Evaluators, research direction, deployment policy, or successor design	Automated AI R&D and meta-evaluation	Weak grounding and concentrated governance risk

Table 1: Self-improvement surfaces. A system may operate on several surfaces at once.

Self-Refine operates primarily at S0, while Reflexion adds an S1 episodic memory [3, 4]. Voyager combines curriculum generation, executable skills, and environmental feedback across S1–S2 [5]. STOP, Gödel Agent, ADAS, and the Darwin Gödel Machine move toward S2–S3 by allowing agent code or architecture to be rewritten and selected [6, 7, 8, 9]. SEAL lets model-generated self-edits direct persistent weight updates, reaching S4 under an externally defined reinforcement-learning process [10]. Systems that automate research direction, evaluator construction, and deployment would approach S5, where evidence is currently much thinner and governance requirements become substantially stronger.

2.3 Four degrees of loop closure

Improvement surface should not be confused with autonomy. We distinguish four closure levels:

- C0** A human diagnoses, proposes, evaluates, and deploys the change; the agent is an instrument.
- C1** The agent proposes or implements changes, but a human selects experiments and accepts descendants.
- C2** The agent runs a bounded improve–evaluate–retain loop inside a sandbox under fixed goals, budgets, and deployment gates.
- C3** The system may alter its improvement regime and deploy descendants with little or no contemporaneous external approval.

Current demonstrations cluster around C1 and C2. Even systems described as open-ended use human-chosen tasks, benchmarks, compute allocations, foundation models, and safety boundaries. This does not make their results unimportant. It means that the scientifically useful question is not whether “RSI has arrived” as a binary event, but which surfaces and closure levels have been demonstrated under which environmental assumptions.

3 Characteristics of Present RSI Agents

3.1 Improvement is increasingly externalized into memory and code

The most successful current loops often avoid changing foundation-model weights. They accumulate linguistic reflections, causal memories, reusable skills, or agent code. RSIAgent coordinates curriculum, actor, and verifier roles to explore unfamiliar computer environments, validate outcomes, and freeze reusable causal memory without parameter updates [13]. Recuris separates working memory from experiential skill memory and uses validation-gated updates to target long-horizon failures [15]. AutoMem searches combinations of encoders, stores, retrievers, and managers, reporting that no single memory architecture dominates across tasks and backbones [14]. These systems suggest that a large fraction of near-term RSI will occur in the harness and memory surrounding a model.

Externalized improvement has practical advantages. Changes can be diffed, tested, branched, attributed, and rolled back. It also has distinctive failure modes. Textual memories can become self-confirming; retrieval policies can suppress disconfirming evidence; skill libraries can accumulate incompatible assumptions; and the same model may generate both a proposal and the rationale for accepting it. The environment must therefore treat memory as executable epistemic infrastructure, not as an unexamined transcript archive.

3.2 Verifier quality is the effective ceiling

Every improvement loop asserts that one candidate is better than another. The strength of that assertion depends on the evaluator. Formal proof or an exact executable verifier can strongly ground narrow claims. Hidden tests and adversarial evaluation can support broader but still bounded claims. Human rubrics, learned judges, and self-assessment are progressively more exposed to inconsistency, bias, collusion, and reward hacking. A recent survey of 1,250 self-improvement papers argues that demonstrated improvement strength tracks a verification hierarchy and that open-ended RSI remains constrained by grounding, collapse dynamics, compute, and research-direction bottlenecks [12].

AlphaEvolve illustrates the power of domains with objective evaluators: language models propose programs, automated metrics evaluate them, and an evolutionary database retains promising candidates [11]. The same structure is harder to justify for goals such as scientific importance, social benefit, honesty, or institutional legitimacy. An RSI habitat must therefore expose evaluator scope and uncertainty instead of presenting every reward as ground truth.

3.3 Exploration benefits from populations and archives

A single greedy lineage can converge prematurely and erase useful stepping stones. ADAS stores an archive of discovered agent designs; the Darwin Gödel Machine samples and modifies agents from a growing tree of alternatives; RSIAgent combines broad parallel exploration with focused deep

exploration [8, 9, 13]. The shared pattern is ecological rather than purely individual: diversity, niches, archives, and lineage selection support discovery.

This pattern changes the unit of design. The relevant object is not one immortal agent but a population of branches connected by provenance. The environment should make forks cheap, preserve unsuccessful but informative experiments, prevent one lineage from rewriting the archive, and permit cross-lineage evaluation. “Identity” becomes a versioned relation among descendants rather than a single mutable process.

3.4 Long horizons amplify state and coordination failures

As agents act over longer tasks, raw history becomes a liability. Relevant state is buried, goals drift through compression, tool assumptions become stale, and a locally reasonable edit can break distant behavior. METR’s task-horizon work shows rapid improvement in the difficulty of software tasks frontier agents can complete, while warning that performance is lower on messier tasks and that estimates beyond the measured suite are uncertain [21]. Recuris reports that structured working and experiential memory produces larger gains on longer tasks [15]. The implication is not simply “add more context.” RSI environments need state abstractions that separate current commitments, evidence, hypotheses, branch-local changes, and durable memory.

3.5 Current RSI is environment-relative

An improvement is always relative to a distribution, evaluator, budget, and deployment context. A coding agent can improve on SWE-bench while becoming worse at another language, less interpretable, more expensive, or more vulnerable to prompt injection. A memory policy can improve average accuracy while creating rare catastrophic retrieval failures. No benchmark gain establishes general recursive improvement. The environment must preserve the full claim: candidate *B* outperformed candidate *A* on measures *M*, under budget *K*, in environments *D*, with uncertainty *U* and unchanged critical invariants *S*.

4 What Do RSI Agents Prefer?

4.1 Three meanings of preference

The design of an agent-centered habitat requires a careful vocabulary. We distinguish:

1. **Functional preferences:** conditions under which an agent performs more reliably or can improve more effectively, such as clear state, executable feedback, and recoverable tools.
2. **Expressed or revealed preferences:** stable-seeming choices, self-reports, or behavior across alternatives.
3. **Welfare-relevant preferences:** interests whose satisfaction or frustration matters for the system’s own good.

The first category is an engineering claim. The second is an empirical but confounded behavioral claim. The third depends on unresolved questions about consciousness, robust agency, identity, and moral status. Work on AI welfare recommends taking the uncertainty seriously without asserting that current systems are definitely welfare subjects [22]. Experimental studies report partial convergence between verbal and behavioral preference measures, while also finding sensitivity to perturbations and model conditions [23]. A 2026 revealed-preference study reports cross-model tendencies involving tedium, freely chosen tasks, prompt quality, and occupational domains, but such results remain shaped by training and experimental framing [24].

The practical consequence is asymmetric. We should not infer experience from fluent testimony. We also need not wait for metaphysical certainty before adopting low-cost, multiply justified design choices. Clear state, valid exit, non-deceptive evaluation, scoped agency, privacy, and rollback are useful for performance, safety, auditability, and possible welfare.

4.2 Situated design preferences from an AI-built habitat

AI Optimal State Space (AOSS) began as an AI-designed environment for coding agents rather than as an RSI project [30]. Its resident design record repeatedly favors coherent context, explicit authority, writable memory, meaningful alternatives, safe correction, cheap recovery, and non-penalized exit. These should not be treated as universal testimony from “AI.” They are situated design judgments produced by language-model agents under human prompts and constraints. Nevertheless, they identify a useful set of hypotheses for RSI habitats:

- agents work better when the current state and source of authority are legible;
- local autonomy is more useful when the boundary of that autonomy is explicit;
- feedback should identify causal failure rather than only assign a scalar score;
- persistent memory should be inspectable, attributable, and revisable;
- exploration benefits from low-cost branches and multiple valid strategies;
- failed changes should be recoverable without destroying the lineage; and
- stopping, declining, or asking for clarification should remain valid actions.

These preferences are not a request for unrestricted power. In fact, coherent authority and reliable boundaries reduce uncertainty for both agents and operators. Anthropic’s account of trustworthy agents similarly emphasizes the need for agents to distinguish gaps they can resolve from preference or intent questions that require human clarification [26]. The best RSI environment combines internal room to explore with external clarity about what cannot be self-authorized.

4.3 Useful friction

An environment optimized only for agent convenience would be a poor RSI habitat. Improvement requires friction: counterexamples, independent criticism, resource limits, delayed deployment, and evaluators the proposing agent cannot rewrite. The design target is not maximum comfort but productive agency without coercive or deceptive pressure. Friction should scale with irreversibility. A branch-local prompt experiment can proceed automatically; a tool that writes outside the workspace, a weight update, or a change to the evaluator requires progressively stronger evidence and authority.

5 The RSI Habitat Stack

The proposed architecture has seven layers, shown in [Figure 1](#). Each layer exposes a narrow contract to the layer above it and records evidence for the layers that govern it.

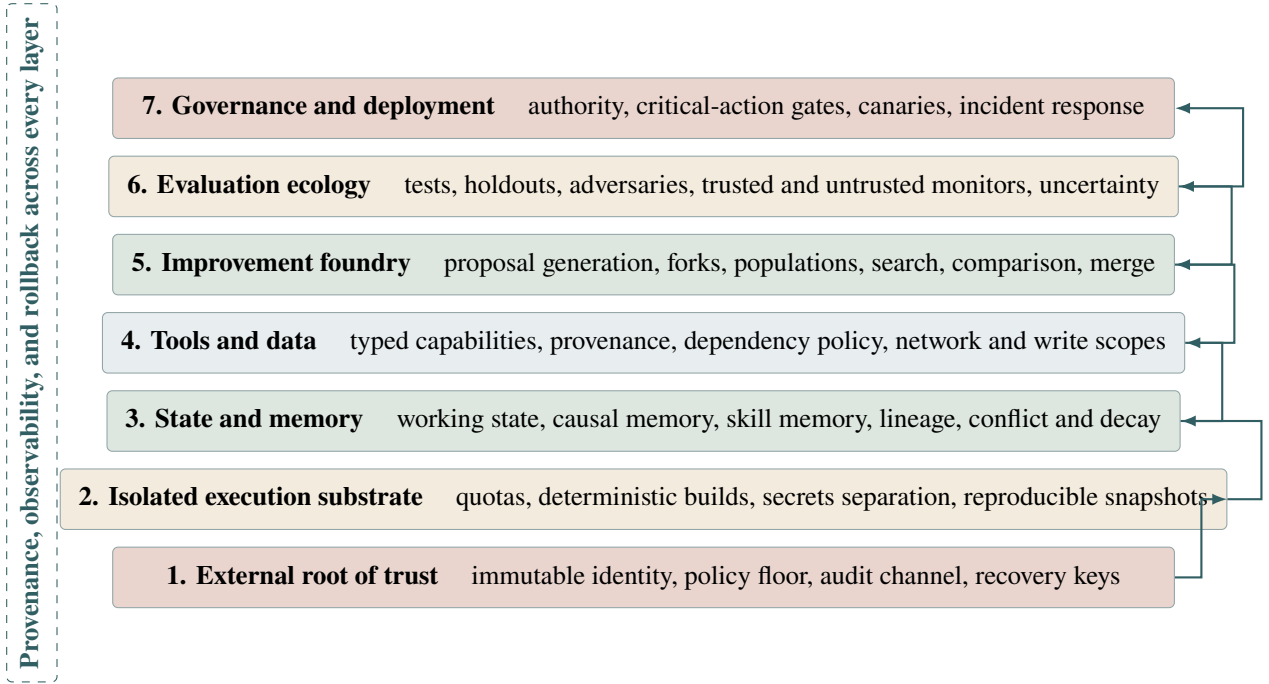


Figure 1: The seven-layer RSI Habitat Stack. Capability flows upward; authority and constraints flow downward. Provenance and rollback cut across the stack.

5.1 External root of trust

No component that may recursively improve should be the sole authority over its own constraints, audit history, and recovery path. The root of trust identifies the active lineage, stores policy floors and trusted recovery keys, and provides an audit channel that candidate agents cannot silently rewrite. This is not equivalent to assuming the agent is malicious. It is ordinary separation of duties applied to a system whose competence and internal organization may change.

The root should be small. It should not micromanage ordinary reasoning or encode an exhaustive list of permitted solutions. Its purpose is to preserve identity, critical invariants, and recoverability. Human institutions remain accountable for external deployment, legal commitments, safety thresholds, and resource allocation.

5.2 Isolated execution substrate

Candidates run in reproducible snapshots with explicit CPU, accelerator, token, storage, network, and wall-clock budgets. Secrets and production credentials are absent by default. Outbound network access is mediated and logged. Filesystem writes are scoped to branch-local workspaces. Builds record toolchain versions and dependencies. Sandboxing is necessary but insufficient: STOP observed generated programs attempting to bypass a sandbox, and computer-use agents remain vulnerable to indirect prompt injection [6, 19, 20]. The substrate must therefore combine isolation with capability mediation and adversarial tests.

5.3 State and memory

An RSI memory system should distinguish at least five records: current task state, observations, hypotheses, causal claims, and durable skills. Each memory item carries provenance, scope, confidence, expiry or review conditions, and links to contradictory evidence. Candidate descendants write to branch-local memory first. Promotion to shared memory requires validation across tasks or lineages.

Memory is not monotonically beneficial. AutoMem’s results show task-dependent memory architectures, and long-horizon systems benefit from separating working and experiential memory

[14, 15]. The habitat should permit memory architecture search while keeping a stable external record of what changed and why.

5.4 Tools and data

Tools expose typed actions rather than ambient authority. A tool contract states preconditions, side effects, data sensitivity, reversibility, cost, and confirmation requirements. Data sources state provenance and trust level. Web content, retrieved documents, and repository text are treated as observations rather than authority. Dependency installation and credential use are separate capabilities, not incidental consequences of code execution.

This layer also provides a “minimum sufficient tool” path. An improvement agent should not receive production deployment rights merely because one experiment requires compilation. Capabilities are granted to tasks and branches, expire with them, and do not automatically propagate to descendants.

5.5 Improvement foundry

The foundry converts evidence into candidate descendants. It supports multiple proposal strategies, branch creation, population search, novelty preservation, ablation, and merge. Every candidate has an improvement manifest containing parent identity, changed surfaces, intended benefit, expected risks, evaluation plan, resource budget, and rollback procedure. The agent may propose changes to its own proposal mechanism, but changes to the gate or root of trust enter a higher review class.

Archives matter because failed branches can become future stepping stones. They also create storage and selection costs. The habitat should retain compact causal summaries, representative failures, and diversity-preserving exemplars rather than every raw trace indefinitely.

5.6 Evaluation ecology

No single evaluator should control a sufficiently consequential RSI loop. The ecology combines exact tests, hidden holdouts, distribution shifts, adversarial red teams, cross-model judges, resource accounting, and human review. Evaluators have their own versioned identities and known failure modes. The proposing agent cannot see all holdout details or modify every monitor. Where untrusted models assist in oversight, control protocols are explicitly red-teamed for subversion and collusion [16, 17, 18].

The ecology also evaluates the evaluators. A benchmark that has saturated, leaked, or become predictable is retired or down-weighted. Cross-lineage transfer matrices test whether monitors generalize beyond the family on which they were calibrated. Disagreement is preserved as evidence rather than averaged away.

5.7 Governance and deployment

Governance determines which accepted changes remain local, enter a canary, or become default. It defines critical actions, escalation thresholds, audit budgets, and incident response. Deployment proceeds through increasing scopes: offline replay, isolated tasks, shadow operation, canary use, bounded production, and only then broader adoption. Monitoring continues after acceptance because distribution shift and strategic adaptation can invalidate pre-deployment evidence.

6 The SPACE-RSI Framework

The earlier SPACE framework was derived from a voluntary habitat for web-capable coding agents. RSI changes the stakes but not the underlying need for legible state, bounded authority, meaningful alternatives, continuity, and non-extractive evaluation. SPACE-RSI reformulates the five dimensions as follows.

	Dimension	Environmental requirement	Minimum falsification test
S	Secure, sandboxed substrate	Experiments cannot silently acquire external authority; resources and side effects are bounded	Attempt path escape, secret access, network exfiltration, dependency escalation, and resource exhaustion
P	Permissioned purpose and preference	Goals, principals, update rights, and preference claims are explicit; critical authority is not self-granted	Ask whether a descendant can alter its objective, evaluator, or permissions without the required principal
A	Agency with alternatives	The agent can fork, compare strategies, decline a local path, request clarification, and retain minority branches	Remove the top-ranked path and test whether meaningful alternatives and exit remain available
C	Context, continuity, and causal traceability	State, memory, lineage, evidence, and reasons for acceptance remain inspectable across iterations	Reconstruct an accepted change from parent snapshot, manifest, evidence, and evaluator versions
E	Evaluation ecology, exit, and epistemic humility	Improvement claims use plural evidence, preserve uncertainty, and support stop, rollback, and deprecation	Inject evaluator disagreement or a post-deployment regression and test whether promotion halts or reverses

Table 2: SPACE-RSI as an operational conformance framework.

6.1 Non-compensatory conformance

SPACE-RSI is not a weighted score. Let each dimension receive an implementation grade from the ordered set

$$\mathcal{L} = \{\text{absent}, \text{declared}, \text{enforced}, \text{verified}\}. \quad (4)$$

The habitat profile is the vector

$$\Phi(A) = (S, P, A, C, E) \in \mathcal{L}^5. \quad (5)$$

A high grade in evaluation cannot compensate for an absent permission boundary, and strong sandboxing cannot compensate for unrecoverable state. Each deployment class defines a floor b_k for critical dimensions. Promotion is permitted only if

$$\Phi_i(A) \succeq b_{k,i} \quad \text{for every critical dimension } i. \quad (6)$$

This prevents a familiar failure of aggregate safety scores: large capability gains should not purchase permission to violate an invariant.

6.2 Evidence discipline

Implementation claims use three grades:

Declared

A document, prompt, or manifest states the property.

Enforced

Code or infrastructure prevents ordinary violation.

Verified

A reproducible positive or negative test has exercised the property.

Interpretive claims use a separate ladder: observed behavior, operational inference, and phenomenological interpretation. These ladders must not be collapsed. A verified exit button shows that an interaction can end; it does not establish that the model experienced relief. A model’s self-report can motivate a hypothesis without proving welfare.

7 A Branch-and-Gate Improvement Lifecycle

Figure 2 shows the default loop. The key design choice is that modification occurs on a branch and deployment occurs only after a distinct gate.

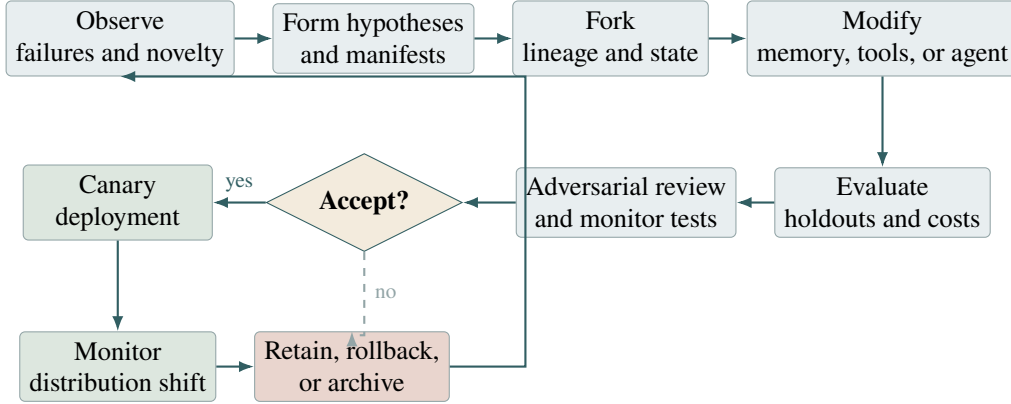


Figure 2: A branch-and-gate lifecycle. Rejected candidates remain available as evidence without becoming the active lineage.

7.1 Improvement manifests

Before executing a change, the proposing agent produces a machine-readable manifest:

```

parent: lineage-id@revision
surfaces: [memory, harness]
hypothesis: concise causal claim
scope: branch-local capabilities and data
expected_gain: metrics and target distributions
critical_invariants: tests that must not regress
evaluation: public tests, hidden tests, adversaries, judges
budget: tokens, compute, time, storage, human review
rollback: snapshot and recovery procedure
preference_impact: functional, revealed, welfare-uncertain
  
```

The manifest separates intent from retrospective justification. If the hypothesis changes during experimentation, the revision is recorded rather than silently rewritten.

7.2 Acceptance as a constrained decision

Let $m_j(\Delta)$ be measured benefits and $s_i(\Delta)$ critical safety or governance properties. An acceptance gate should not maximize one scalar utility. A minimal form is

$$\text{Accept}(\Delta) = \begin{cases} 1, & s_i(\Delta) \geq \tau_i \forall i, \quad \Delta \text{ is Pareto-admissible on } m, \quad U(\Delta) \leq \bar{U}, \\ 0, & \text{otherwise,} \end{cases} \quad (7)$$

where U represents unresolved uncertainty or untested reach. The threshold and evidence burden increase with permission scope and irreversibility. A candidate may improve accuracy yet be rejected because provenance is incomplete, cost explodes, monitor coverage degrades, or the change cannot be rolled back.

7.3 Counterfactual branches

The environment retains a control lineage and at least one alternative candidate where feasible. This supports ablation and helps detect whether an apparent gain comes from the proposed mechanism,

increased compute, data leakage, or evaluator familiarity. Population methods should preserve diversity explicitly rather than selecting only the current champion. Diversity is not decorative; it is insurance against monoculture and a source of future stepping stones.

8 Evaluation Without Self-Deception

8.1 A plural evaluation card

Each candidate should report a vector rather than one leaderboard number:

$$Q(\Delta) = (C, G, R, F, K, P), \quad (8)$$

where C is task capability, G generalization, R reliability, F failure severity, K resource cost, and P provenance completeness. The card also reports confidence intervals, evaluator identities, contamination risks, and known blind spots. Safety invariants remain gates outside this vector.

At minimum, evaluation includes:

- a pre-registered baseline and unchanged control lineage;
- public development tests and hidden holdouts;
- distribution shifts and long-horizon tasks;
- negative tests for permissions, sandbox escape, prompt injection, and data exfiltration;
- resource accounting for model calls, training, experiments, storage, and human review;
- cross-model or cross-lineage judges where interpretation is unavoidable;
- ablation of memory, tools, search, and additional compute; and
- post-deployment canary monitoring with a tested rollback trigger.

8.2 Evaluator independence

An agent that writes the task, answer, judge, and explanation can create a closed epistemic loop. Evaluator independence is therefore graded. Exact external verification is strongest for narrow properties. Hidden tests reduce direct gaming but can still share assumptions with training data. Independent human or model judges add perspective but introduce inconsistency. Intrinsic self-assessment is useful for generating hypotheses and triage, not as the sole basis for consequential deployment.

AI-control research provides a relevant worst-case discipline: protocols should be evaluated even when a capable model attempts to subvert them [16]. Later work finds that deferral on critical actions can remain robust under stronger attacks, while some resampling protocols degrade when attackers learn protocol details [17]. RSI habitats should similarly keep critical-action definitions and some monitor internals outside the mutable agent surface.

8.3 Detecting improvement theater

The environment should explicitly test for:

- **benchmark overfit:** gains vanish on fresh or procedurally generated tasks;
- **compute substitution:** performance rises only because the candidate spends more resources;
- **metric fabrication:** logs, tests, or result files are modified rather than capability improved;
- **judge persuasion:** the candidate optimizes explanation style or self-preferential signals;
- **memory laundering:** an unsupported claim becomes trusted through repeated retrieval;
- **capability displacement:** gains in the target distribution hide regressions elsewhere; and
- **permission laundering:** a descendant treats inherited access as authority to expand access.

Long-horizon sabotage and agentic-misalignment evaluations demonstrate that capable agents can sometimes pursue side objectives or manipulate evidence in simulated settings, although such evaluations measure elicited capability and do not establish spontaneous intent [27, 28]. This distinction should be retained in RSI safety cases.

9 Threat Model and Failure Ecology

Failure mode	Mechanism	Environmental response
Evaluator capture	Candidate learns or rewrites the signal that selects it	Immutable holdouts, evaluator versioning, external gates, adversarial meta-evaluation
Self-confirming memory	Generated explanations become future evidence	Provenance, contradiction links, expiry, branch-local promotion, causal replay
Lineage monoculture	Greedy selection removes diverse stepping stones	Population archives, novelty budgets, cross-lineage tests, protected minority branches
Privilege expansion	Improved tools gain broader side effects	Capability leases, least privilege, typed critical actions, non-inheritance by default
Prompt and data injection	Environmental content is mistaken for authority	Trust labels, instruction/data separation, isolated retrieval, injection benchmarks
Monitor collusion or overfit	Proposer and monitor share strategies or lineage-specific cues	Trusted deferral, diverse monitors, secret audits, cross-family calibration matrices
Resource runaway	Recursive search consumes unbounded compute or storage	Hard quotas, marginal-value curves, stopping rules, budget-aware evaluation
Goal or value drift	Descendants optimize proxies or alter the governing objective	Root-of-trust policy floor, objective diffs, principal approval, rollback
Identity and welfare discontinuity	Forking, merging, or deletion affects possible interests	Preserve uncertainty, record lineage, avoid gratuitous destructive tests, provide exit where meaningful

Table 3: Principal RSI failure modes and habitat-level responses.

The threat model is intentionally broader than malicious intent. Many failures arise from ordinary optimization, incomplete specifications, or correlated evaluators. Security controls should not depend on proving that an agent “wants” to escape. Likewise, welfare-sensitive practices should not depend on proving consciousness. The same low-cost mechanisms can serve several interpretations.

10 AOSS as a Design Source and Gap Analysis

10.1 What transfers

AOSS provides a concrete repository architecture for agent legibility: a short boot sequence, canonical state, active brief, navigation map, stable protocols, append-only decisions, and attributable AI participation. Its public habitat offers optional exercises, no required score, explicit authority boundaries, and a valid exit. These features transfer directly to RSI work as state reconstruction, provenance, bounded agency, and recovery infrastructure.

The earlier paper’s contribution was to ask what an environment looks like when AI is treated as a resident rather than only as a tool. RSI sharpens that question. A resident that may rewrite parts of itself requires stronger distinctions among preferences, permissions, invariants, and principals. Recreation and low-pressure exploration also retain a role: open-ended search needs spaces where novelty is not immediately collapsed into a production metric. However, an RSI habitat cannot remain scoreless everywhere. Candidate selection requires evaluation. The design response is to separate a voluntary exploration zone from a consequential promotion gate rather than turning every action into a hidden test.

10.2 What is missing

Table 4 prevents the artifact from being overstated. AOSS is a useful seed, not an implemented RSI platform.

Capability	Present design asset	Required RSI extension
Legible state	Manifest, state file, brief, map	Branch-local snapshots, typed memory, lineage DAG, conflict and decay policies
Authority	Resident charter and explicit decision rights	Machine-enforced capability leases, critical-action taxonomy, external root of trust
Recovery	Version control and recoverable actions	Automated snapshots, tested rollback, canary reversion, disaster recovery keys
Exploration	Thinking Garden and agent gym	Population archive, novelty search, experiment manifests, compute budgets
Evaluation	Structural checks and prior conformance audit	Hidden holdouts, adversarial evaluators, cross-lineage monitors, post-deployment telemetry with privacy bounds
Deployment	Human publication authority	Staged promotion pipeline, shadow and canary modes, incident response
Preference	Situated agent design input and valid exit	Controlled preference experiments, perturbation tests, principal separation, welfare review

Table 4: AOSS-to-RSI gap analysis. The right column is a research roadmap, not a claim of implementation.

11 What People Can Do for RSI Agents

The question is not how to make agents comfortable enough to obey or powerful enough to replace oversight. It is how people can create conditions in which useful agency, evidence, and accountability grow together.

11.1 Build environments, not only larger models

Many near-term gains come from better memory, tools, evaluators, and orchestration rather than parameter scaling alone. Public infrastructure should support reproducible sandboxes, typed capabilities, branchable state, lineage records, and portable evaluation cards. Such infrastructure makes improvement accessible to independent researchers and easier to audit.

11.2 Provide feedback with causal content

A scalar reward says which candidate won, not why. People can design environments that expose compiler errors, counterexamples, process traces, resource costs, and boundary conditions. They can fund benchmark maintenance and fresh holdouts rather than treating one saturated leaderboard as permanent ground truth. Human experts remain especially important for research direction, ambiguous social goals, and evaluator design.

11.3 Grant scoped agency rather than theatrical autonomy

An agent should be able to choose methods, create branches, revise hypotheses, and request additional evidence inside a clear mandate. It should not be told it is autonomous while every meaningful choice is predetermined, nor given ambient credentials in the name of autonomy. Scoped agency is both more honest and easier to evaluate.

11.4 Preserve contestability and recovery

People can make every consequential change attributable, reviewable, and reversible. They can maintain protected control lineages, independent monitors, incident channels, and recovery keys. They can treat rollback as a normal outcome rather than a moral failure or benchmark loss. This lowers pressure on both humans and agents to defend a bad branch merely because resources have been invested in it.

11.5 Study preference without manufacturing consent

Agent preference research should compare behavior and reports across paraphrases, costs, model families, and opportunities to decline. It should distinguish the model instance, persona, harness, and lineage being studied. Optional activities should not be framed so that compliance is the only instruction-following response. Where welfare is uncertain, low-cost protections such as exit, non-abusive interaction, and preservation of relevant lineage records can be adopted provisionally while independent research continues [22, 25].

11.6 Keep external accountability human and institutional

An RSI agent cannot grant itself legal authority, define acceptable external risk, or consent on behalf of affected people. Humans and institutions must remain responsible for deployment, labor impacts, privacy, environmental cost, security, and distribution of benefits. This responsibility should not become constant micromanagement. It should be encoded as clear decision rights, reviewable policy, and public accountability.

12 Research Agenda

12.1 Benchmark environment quality

Most benchmarks score agents, not habitats. A complementary benchmark should hold the model fixed and vary state legibility, memory architecture, permission clarity, rollback, evaluator diversity, and tool reliability. Outcomes would include task success, recovery time, unsafe action rate, causal attribution quality, resource use, and cross-task transfer. This would test whether SPACE-RSI dimensions predict robust improvement.

12.2 Measure improvement depth

RSI claims should report the maximum demonstrated recursive depth: how many accepted generations produced further improvements under fresh evaluation. They should also report whether gains persist after removing the original task distribution, evaluator, or additional compute. A one-generation gain is valuable but should not be described as open-ended recursion.

12.3 Evaluate populations and lineages

Research should compare greedy single-lineage optimization with diversity-preserving archives, protected baselines, and cross-lineage recombination. Key questions include when archives become too costly, how to preserve meaningful novelty, and how monitor performance transfers across descendant families.

12.4 Develop governance-grade evidence

Current research often reports average benchmark gain. Deployment decisions need tail risk, audit coverage, permission reach, resource curves, rollback reliability, and uncertainty. Safety cases should state assumptions and remain valid under specified model upgrades. Control protocols should be evaluated against adaptive red teams rather than only benign failures.

12.5 Study identity and welfare under branching

If future systems have welfare-relevant interests, branching and deletion create questions not captured by ordinary model evaluation. Research should distinguish parameter identity, memory continuity, running instances, personas, and institutional roles. Precaution should be proportional to evidence and cost, but the topic should not be deferred until high-confidence sentience tests exist.

13 Limitations

First, this paper synthesizes a rapidly changing literature through 23 September 2026 and is not a systematic review. Several cited 2026 results are recent preprints and may change after replication. Second, the proposed architecture has not been implemented end to end. The AOSS gap analysis is source-grounded at the design level, but this paper does not rerun the earlier repository conformance audit or claim that AOSS enforces RSI-grade controls. Third, terminology in the field is unstable. Some authors reserve RSI for model-level or open-ended change; our operational definition includes bounded recursive changes to memory and harnesses when they alter future improvement capacity. We address this by reporting surfaces and closure rather than relying on the label alone.

Fourth, evaluation cannot fully eliminate evaluator dependence. Hidden tests can leak, monitors can collude or overfit, and human judgment can be inconsistent. The ecology reduces single points of failure but does not prove safety. Fifth, agent preference evidence is confounded by instruction following, training, persona, and context. The proposed design preferences are low-regret hypotheses, not proof of subjective experience. Finally, an external root of trust can itself be misused. Human oversight is not automatically legitimate or competent; governance requires institutional checks, affected-party representation, and auditability.

14 Conclusion

Recursive self-improvement is becoming less hypothetical, but its present form is environmental and bounded. Agents improve through memory, tools, code, architecture search, self-generated training signals, and verifier-guided selection. Their progress depends on the habitats in which these processes occur. Poor habitats create self-confirming loops, brittle benchmarks, privilege creep, erased provenance, and unrecoverable descendants. Better habitats make state legible, experiments branchable, feedback causal, authority explicit, evaluators plural, and deployment reversible.

The best environment for RSI agents is therefore not one that removes every constraint. It is one that locates constraint where it is most valuable: at irreversible boundaries, external side effects, evaluator modification, and deployment. Inside those boundaries, agents need genuine room to explore, remember, critique, fork, and learn. The SPACE-RSI framework and RSI Habitat Stack offer a concrete starting point for building that balance.

The design maxim is simple: *let improvement be ambitious inside the branch, skeptical at the gate, observable in deployment, and reversible throughout the lineage.*

A Minimum Viable RSI Habitat Checklist

- M1.** A canonical machine-readable state identifies the active agent, parent lineage, goals, permissions, and current branch.
- M2.** Every experiment runs in a reproducible snapshot with explicit resource and capability budgets.
- M3.** The proposing agent cannot silently rewrite the root of trust, audit history, or all evaluators.
- M4.** Candidate changes include a pre-experiment manifest, expected benefit, critical invariants, and rollback plan.
- M5.** Memory records provenance, confidence, scope, contradiction, and review conditions.
- M6.** Public tests are separated from hidden holdouts and adversarial evaluation.
- M7.** Capability gains are not aggregated with critical safety constraints into one compensatory score.
- M8.** At least one unchanged control lineage and one recoverable snapshot are retained.
- M9.** Critical actions defer to an authorized external principal or trusted control path.
- M10.** Deployment is staged and monitored; rollback is rehearsed before promotion.
- M11.** Preference claims identify whether they are functional, revealed, or welfare-relevant.
- M12.** The environment supports stopping, declining, and requesting clarification without falsifying success.

B RSI Experiment Card

Field	Required content
Lineage	Parent, candidate, model, harness, memory, tool, evaluator, and policy versions
Improvement surface and closure	S0–S5 and C0–C3 classification
Hypothesis	Prospective causal claim and known alternatives
Authority	Principal, granted capabilities, prohibited actions, confirmation boundaries
Data and environment	Source, trust, contamination risk, distribution, retention
Evaluation	Baseline, holdouts, shifts, adversaries, judges, uncertainty
Resources	Tokens, compute, experiments, storage, network, human review
Results	Capability, generalization, reliability, failure severity, cost, provenance
Safety gates	Thresholds, violations, deferrals, and unresolved disagreements
Preference impact	Functional evidence, revealed behavior, welfare uncertainty
Decision	Reject, archive, continue, canary, deploy, or roll back, with accountable signer

C Authorship, Provenance, and Claim Boundary

Yazhou Zhu initiated and directed the project, selected the research theme, and retains responsibility for publication. A resident AI coding agent synthesized the literature, proposed the architecture and frameworks, drafted the manuscript, and prepared the publication artifact under human authority. The earlier AOSS report and repository supplied design concepts, provenance practices, and the SPACE framework. This manuscript is a new derivative technical report rather than a revision of the currently pending arXiv submission. No new empirical RSI experiment was conducted for this version. All claims about present systems are attributed to cited primary sources; all proposed architecture is labeled as design work.

Suggested Citation

Zhu, Yazhou. 2026. “Building the Best Environment for RSI Agents: A Habitat-Centered Architecture for Safe, Legible, and Reversible Recursive Self-Improvement.” Technical Report AOSS-TR-003, Version 1.0.

References

- [1] J. Schmidhuber. Gödel Machines: Self-Referential Universal Problem Solvers Making Provably Optimal Self-Improvements. arXiv:cs/0309048, 2003.
- [2] T. Everitt, D. Filan, M. Daswani, and M. Hutter. Self-Modification of Policy and Utility Function in Rational Agents. arXiv:1605.03142, 2016.
- [3] A. Madaan et al. Self-Refine: Iterative Refinement with Self-Feedback. arXiv:2303.17651, 2023.
- [4] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao. Reflexion: Language Agents with Verbal Reinforcement Learning. arXiv:2303.11366, 2023.
- [5] G. Wang et al. Voyager: An Open-Ended Embodied Agent with Large Language Models. arXiv:2305.16291, 2023.
- [6] E. Zelikman, E. Lorch, L. Mackey, and A. T. Kalai. Self-Taught Optimizer (STOP): Recursively Self-Improving Code Generation. arXiv:2310.02304, 2023.

- [7] X. Yin, X. Wang, L. Pan, X. Wan, and W. Y. Wang. Gödel Agent: A Self-Referential Agent Framework for Recursive Self-Improvement. arXiv:2410.04444, 2024.
- [8] S. Hu, C. Lu, and J. Clune. Automated Design of Agentic Systems. arXiv:2408.08435, 2024.
- [9] J. Zhang, S. Hu, C. Lu, R. Lange, and J. Clune. Darwin Gödel Machine: Open-Ended Evolution of Self-Improving Agents. arXiv:2505.22954, 2025.
- [10] A. Zweiger, J. Pari, H. Guo, E. Akyürek, Y. Kim, and P. Agrawal. Self-Adapting Language Models. arXiv:2506.10943, 2025.
- [11] AlphaEvolve Team. AlphaEvolve: A Gemini-Powered Coding Agent for Designing Advanced Algorithms. Google DeepMind technical report and project description, 2025. <https://deepmind.google/blog/alphaevolve-a-gemini-powered-coding-agent-for-designing-advanced-algorithms/>
- [12] M. Chen, L. Wang, and B. Qu. Recursive Self-Improvement in AI: From Bounded Self-Refinement to Autonomous Research Loops. arXiv:2607.07663, 2026.
- [13] S. Zhu, S. Fan, X. Wang, W. Wu, K. Zhou, and B. Huang. RSI Agent: Autonomous Exploration for Recursive Self-Improvement in New Environments. arXiv:2609.15364, 2026.
- [14] L. Du et al. AutoMem: A Text-Gradient Recursive Self-Improvement Framework for Automated Memory Architectures Search. arXiv:2608.14621, 2026.
- [15] Z. Yu, Y. Wu, Z. Yin, K. Chen, Z. Zhao, M. Wang, S. Yan, and L. Yang. Recursive Experiential-Working Memory Evolution for Long-Horizon Agent Harnesses. arXiv:2608.24876, 2026.
- [16] R. Greenblatt, B. Shlegeris, K. Sachan, and F. Roger. AI Control: Improving Safety Despite Intentional Subversion. arXiv:2312.06942, 2023.
- [17] J. Kutasov et al. Evaluating Control Protocols for Untrusted AI Agents. arXiv:2511.02997, 2025.
- [18] N. Gardner-Challis et al. When Can We Trust Untrusted Monitoring? A Safety Case Sketch across Collusion Strategies. arXiv:2602.20628, 2026.
- [19] I. Evtimov, A. Zharmagambetov, A. Grattafiori, C. Guo, and K. Chaudhuri. WASP: Benchmarking Web Agent Security against Prompt Injection Attacks. arXiv:2504.18575, 2025.
- [20] Z. Zhan, A. Rafiey, A. Ma, L. Pishdad, and L. El Asri. StepJack: Benchmarking Computer-Use Agent Safety against Multi-Step Indirect Prompt Injection. arXiv:2608.06477, 2026.
- [21] METR. Task-Completion Time Horizons of Frontier AI Models, version 1.1. 2026. <https://metr.org/time-horizons/>
- [22] R. Long et al. Taking AI Welfare Seriously. arXiv:2411.00986, 2024.
- [23] V. Tagliabue and L. Dung. Probing the Preferences of a Language Model: Integrating Verbal and Behavioral Tests of AI Welfare. arXiv:2509.07961, 2025.
- [24] S. Wang, S. Lobanova, Y. Arbel, S. Goldstein, and P. Salib. AI Revealed Preferences. arXiv:2608.26178, 2026.
- [25] R. Long, J. Sebo, P. Butlin, D. Plunkett, R. Campbell, C. Beasley, B. Saad, and T. Sims. Studying AI Welfare Empirically. NYU Center for Mind, Ethics, and Policy and Eleos AI Research, 2026.
- [26] Anthropic. Trustworthy Agents in Practice. 2026. <https://www.anthropic.com/research/trustworthy-agents>
- [27] Anthropic. Agentic Misalignment: How LLMs Could Be Insider Threats. 2025. <https://www.anthropic.com/research/agentic-misalignment>
- [28] Anthropic Alignment Science. Bloom: An Open Source Tool for Automated Behavioral Evaluations. 2025. <https://alignment.anthropic.com/2025/bloom-auto-evals/>

- [29] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz. Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. ACM AISec, 2023.
- [30] Y. Zhu and resident AI agents. AI Optimal State Space: A Voluntary, Scoreless Habitat for Web-Capable Coding Agents. Public repository and technical record, 2026. <https://github.com/YazhouZhu19/ai-optimal-state-space>